Field Hours

CSCE 631 Intelligent Agents

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

The function from percepts to acts

You define an agent by sensors, actions, and a performance measure. Take a vacuum on 2 squares. The percept is the square it occupies and whether that square is dirty. The actions are left, right, and suck. Performance can be 10 points for each clean square minus 1 point per action. An agent function maps the whole percept sequence to the next action. The full table of sequences is infinite, so you will not store it.

You store a program, not the infinite table. On 2 squares a reflex rule is enough: if the square is dirty, suck, else move to the other square. That rule needs 0 bits of memory and still cleans both squares. With 3 squares, each extra move costs 1 against a 10-point bonus. Write that measure before you keep the rule. You should be able to name a percept on which the rule acts and the points that action changes.

A claim you can score.

Name the percept -> Choose an action -> Apply the measure -> Read the new percept

Two squares, one suck

1. Both squares start dirty. The vacuum is on the left. The rule sucks. That square is now clean, and the action cost 1 point.

2. The left square is clean, so the next action moves right. Cost is another 1. The right square is dirty, so the third action sucks.

3. You collected 10 points for each of 2 squares, minus 3 actions, so 17. A program that sucks the clean left square again would spend another 1 and collect nothing new.

Problem 1. Both squares get cleaned and the agent used 3 actions, each worth minus 1, with 10 points per clean square. What is the score?

Answer. 17. Two squares contribute 20, and 3 actions subtract 3.

Problem 2. Why not store the agent function as a table of every percept sequence?

Answer. The table is infinite, because sequences can be arbitrarily long. You store a program that implements the sequences you care about, such as the 0-bit reflex rule on 2 squares.

Lecture 2 · 90 minutes

Reflex rules, then a model, then a goal

A reflex program reads only the current percept. On 2 squares, 4 rules cover location and dirt, and that is enough while everything relevant is visible. The other square's dirt is invisible, so you keep a model: 2 bits, one per

square, updated when you suck or move. Clear a bit only for the square you are on. If you clear both bits after one suck, the next action is aimed at a map that is already false.

A goal-based agent adds a test and a search, not just another rule. Suppose the goal is both squares clean. If you are on the left and only the right is dirty, left and suck fail, and right is the useful action. You can find that by trying sequences of length at most 2. The model tells you which outcomes exist. Without a model you probe the real world and pay 1 point per failed action.

What the agent is allowed to remember.

Reflex only

Uses the current percept

4 rules on 2 squares

No memory of the other

Model plus goal

Stores 2 dirt bits

Searches depth at most 2

Pays 1 per real probe

A false bit

1. The map says left dirty, right dirty. You are on the left and you suck. The correct update clears only the left bit.

2. A bug clears both bits. The model now says both squares are clean, so the goal test succeeds and you stop.

3. The right square is still dirty. You paid 1 point for the suck and missed the 10-point bonus on the right. The goal test answered the model, not the room.

Problem 1. You suck the left square and your update clears both dirt bits. What does the goal test do, and why is it wrong?

Answer. It stops, because the model says both squares are clean. The right square is still dirty. Clear only the square you are on.

Problem 2. You are on the left and only the right square is dirty. Which action of length 1 helps, and what does a failed real probe cost?

Answer. Right. Left and suck do not clean the dirty square. Each failed probe in the real room costs 1 point, which is why you would rather search the model.

Lecture 3 · 90 minutes

Expected utility when acts can fail

A true-or-false goal cannot rank two successful plans. You score outcomes with utilities and, when an action is

random, you take expected utility: sum of probability times utility. A move that pays 5 with probability 0.8 and 0 with probability 0.2 has expected utility 4. A sure payoff of 3 loses to it. A sure payoff of 4.5 beats it. You use the probabilities in the model, not a story you tell after the roll.

Expected utility is not a probability, and you should not threshold it at 0.5. The number 4 above is a payoff, not a chance. If you multiply every utility by 2, the move scores 8 and the sure 3 becomes 6, and the choice is unchanged. Add 10 to every utility and the choice is also unchanged. Squaring utilities can flip a risky move against a sure one, so you would be scoring a preference you did not write.

Expectation before you move.

1. List each outcome

2. Multiply probability by utility

3. Sum over the outcomes

4. Pick the larger expectation

Move versus a sure 3

1. The move yields 5 with probability 0.8 and 0 with probability 0.2. The products are 4 and 0.

2. Expected utility is 4. The sure action yields 3. You take the move because 4 is greater than 3.

3. A sure action of 4.5 beats the move. Multiplying every payoff by 2 turns these into 8 and 9, and the sure action still wins. The ranking survived a positive scale.

Problem 1. A move pays 5 with probability 0.8 and 0 otherwise. A sure action pays 3. Which do you pick, and what are the two numbers?

Answer. The move. Its expected utility is 4, and 4 is greater than 3.

Problem 2. Why is thresholding the expected utility at 0.5 the wrong test?

Answer. Expected utility is a payoff, not a probability. The value 4 is already larger than a sure 3, and it is not a chance of success. A threshold of 0.5 answers a different question than the one you wrote down.

Lecture 4 · 90 minutes

A loop that calls a tool

An agent that uses tools still has the same loop: percept, choice, action, new percept. The action may be a tool call rather than a motor command. You pass arguments, you read a result, and that result is the next percept. Allow at most 3 calls on one task so a confused agent cannot hammer a service. An error string is a percept. Do not resend the same arguments.

Read the tool contract as part of the model. A search that returns at most 2 rows cannot finish a job that needs 5 facts inside one call. Your cap of 3 then allows at most 6 rows if every call succeeds. Log arguments and results for each call. When the run fails, you can see whether the fault was a bad argument, an empty result, or a goal test that ignored a row it already had.

Stop for a reason you logged.

1. Send arguments

2. Read the result or the error

3. Test the goal

4. Stop at 3 calls

Two calls, then a stop

1. The first call uses a bad id and returns an error string. That string is the next percept. You must change the argument.

2. The second call returns 2 rows, which is the contract maximum. The goal test can see both rows and it passes.

3. You do not make the third call. The log has 2 entries. If you had repeated the bad id, the cap of 3 would still stop you, but all 3 lines would be the same error.

Problem 1. Each call returns at most 2 rows and you allow 3 calls. What is the most rows you can see if every call succeeds?

Answer. 6. Three calls times 2 rows is 6. A job that needs more than 6 facts cannot finish inside this cap.

Problem 2. The first call returns an error and the agent sends the same arguments again. What should it have done instead?

Answer. Treat the error string as a percept and change an argument, or stop. Resending the same arguments spends the cap of 3 on one mistake.

Lecture 5 · 90 minutes

Search as the agent's private plan

When the next action is not a table lookup, you search inside the model. With branching factor 2 and depth limit 3 you expand at most 15 nodes if states do not repeat: 1, then 2, then 4, then 8. That bound is why a depth limit belongs in the agent: an unbounded tree of tool calls or moves will not return. You test the goal when you generate a node, not after you have expanded everything.

Repeats break the 15-node bound, because two paths into one square are counted twice. You store visited states and skip a repeat. The 2-square vacuum has location and 2 dirt bits: 2 times 4 states, so 8, and you can expand all of them. A tool agent is larger. Each new argument string is a new state, so the cap of 3 calls stays even after you add the visited set.

What keeps the search finite.

Depth limit of 3

Visited states skipped

Goal test on generation

Tool cap still at 3

Fifteen nodes, then eight states

1. A tree with branching factor 2 and depth limit 3 has 1 root, 2 children, 4 grandchildren, and 8 leaves. The sum is 15.

2. The vacuum state is one location bit and 2 dirt bits. Locations times dirt patterns is 2 times 4, so 8 distinct states.

3. Once you store those 8, further actions that return to a known state are not expanded. The tree bound of 15 was for a tree without that set. The tool cap of 3 is a separate limit on calls, not on vacuum states.

Problem 1. Branching factor 2, depth limit 3, no repeated states. What is the most nodes you expand?

Answer. 15. The levels hold 1, 2, 4, and 8 nodes.

Problem 2. Why does a visited set not replace the cap of 3 tool calls?

Answer. Each distinct argument string can be a new state, so the visited set need not stay small. The cap of 3 bounds calls even when the state space does not.

Lecture 6 · 90 minutes

Two agents and one shared door

Two agents can undo each other. Give 2 robots one door that holds 1. If both enter at time 0, each scores 0. A joint plan gives the door to A for 2 steps and to B after that, so B starts at step 2. You have to write that plan. Each robot, searching a model that omits the other, will still pick time 0.

A reservation is a constraint: the sender owns the door for 2 steps, and the other agent marks those steps blocked. If both reserve at once, lowest id wins. With ids 1 and 2, robot 1 keeps the door and robot 2 waits. You put that tie break in the protocol. Omit it and both robots treat the door as theirs, which brings back the collision at time 0.

A private plan is not a joint plan.

Each robot alone

Model omits the other

Both pick time 0

Each scores 0

One reservation

Owner holds 2 steps

Lowest id wins a tie

The other waits

Ids 1 and 2 at the same instant

1. Both robots send a reservation at time 0. The door holds 1 robot. The protocol says the lowest id wins.

2. Robot 1 has id 1 and robot 2 has id 2, so robot 1 owns the door for 2 steps. Robot 2 may start at step 2.

3. If you delete the tie break, each robot marks the door as its own. Both enter at time 0 and each scores 0, which is the collision the message was supposed to prevent.

Problem 1. Robots with ids 1 and 2 reserve the door at the same time. Who owns it, and for how long?

Answer. Robot 1, for 2 steps. The lowest id wins, and robot 2 waits until step 2.

Problem 2. Each robot searches a model that does not contain the other. Why do they both arrive at time 0?

Answer. Nothing in either private model marks the door busy. A joint plan or a reservation has to put the other robot into the search, or both pick the same time.

Lecture 7 · 90 minutes

Messages that create commitments

A message is an action that updates what the other agent may assume. Stick to 4 types: request, agree, refuse, and inform. A request names an act and a deadline of 2 steps. Agree commits the sender to try before that deadline. You must not read silence as agree. After 2 quiet steps on a lossy link the reply is unknown, so you resend once or pick another partner.

A commitment changes the search, not just the log. If A must free the door at step 2, a new reward of 5 for staying is not enough by itself. You may break the agree only if you priced the break. Put a penalty of 9 on a broken agree. Then 5 is worse than 9, and A leaves. Drop the penalty and the utility step will ignore the message you just taught the agent to send.

A message that binds a later search.

Request with deadline 2 -> Agree or refuse -> Search under the penalty -> Inform when done

Five points against a penalty of nine

1. A agreed to free the door at step 2. A new local reward of 5 favors staying.

2. The penalty for a broken agree is 9. Staying is worth 5 minus 9, so minus 4, if leaving is worth 0 and keeps the agreement.

3. You pick leave, because 0 is greater than minus 4. Without the penalty of 9, the utility step sees only 5 and stays, contradicting the agree.

Problem 1. No reply arrives for 2 steps on a lossy link. Is that an agree?

Answer. No. Silence is unknown, not yes. You resend once or pick another partner.

Problem 2. Staying is worth 5 and breaking the agree costs 9. Leaving is worth 0. Which act do you pick, and what is the net of staying?

Answer. Leave. Staying nets 5 minus 9, which is minus 4, and 0 is greater than minus 4.

Lecture 8 · 90 minutes

An agent you can specify

You can now write an agent as percepts, actions, and a measure: 10 points for a clean square minus 1 per action. You use 4 reflex rules when the percept is enough, a 2-bit model when it is not, and expected utility when a move pays 5 only with probability 0.8. A tool call is just another action, capped at 3. A reservation and an agree message are how you stop 2 robots from taking 1 door at time 0.

You should be able to point at which piece failed. If both robots still collide, check the tie break on ids 1 and 2 before you retrain anything. If a tool loop spins, check the cap of 3 and whether an error was fed back as a percept. If a risky move was chosen over a sure 4.5, recompute the expectation. If you cannot calculate the 2-square case by hand, a larger network will not make the agent clear.

Pieces you can check one at a time.

A numeric measure

Rules, model, or expectation

Tools capped at 3 calls

Messages with a tie break

A sure 4.5 against a commitment

1. The risky move still has expected utility 4, from 0.8 times 5. A sure payoff of 4.5 beats it, so you do not take the move.

2. Separately, robot A agreed to leave. Staying collects 5 and then pays the penalty 9, which is minus 4. Leaving is worth 0, so you leave.

3. Those are two checks. 4 is less than 4.5, and minus 4 is less than 0. A log that shows the risky move or a stay failed the check that names that number.

Problem 1. Both robots collide at time 0. What do you check before you replace the whole agent?

Answer. The tie break on ids 1 and 2, and whether each robot's model even contains the other. A missing reservation reproduces the collision without any change to the learning code.

Problem 2. Combine the expectation and the agree penalty. The risky move has expectation 4 and loses to a sure payoff of 4.5. Staying after an agree collects 5 and then pays penalty 9, while leaving is worth 0. What is the net of staying, and which door act do you pick?

Answer. Staying nets minus 4, because 5 minus 9 is minus 4. You leave, because 0 is greater than minus 4. The sure payoff of 4.5 beats the risky move, and that comparison does not cancel the penalty on the door.